Week 14 Report

Computational Statistics for Data Science · ISS507700

Student ID: 112006234

May 31, 2026

I have completed most of the work by myself, but I used AI tools to help with debugging, understanding some concepts, and checking for typos.

112006202 helped understand how to structure the k-fold cross-validation function in Q3(a)

Q0 · Setup

```
# Load packages
library(rpart)

# Load data
cars <- read.table("auto-data.txt", header = FALSE, na.strings = "?")

# Name columns
names(cars) <- c(
  "mpg", "cylinders", "displacement", "horsepower", "weight",
  "acceleration", "model_year", "origin", "car_name"
)

# Remove car name because it is text and not useful for these models
cars$car_name <- NULL

# Remove rows with missing values
cars <- na.omit(cars)

# Shuffle rows before train/test split and CV
set.seed(27935752)
cars <- cars[sample(1:nrow(cars)), ]

# Model formulas
cars_full <- mpg ~ cylinders + displacement + horsepower + weight +
  acceleration + model_year + factor(origin)

cars_reduced <- mpg ~ weight + acceleration + model_year + factor(origin)

cars_full_poly2 <- mpg ~ poly(cylinders, 2) + poly(displacement, 2) +
  poly(horsepower, 2) + poly(weight, 2) + poly(acceleration, 2) +
  model_year + factor(origin)
```

```
cars_reduced_poly2 <- mpg ~ poly(weight, 2) + poly(acceleration, 2) +
  model_year + factor(origin)

cars_reduced_poly6 <- mpg ~ poly(weight, 6) + poly(acceleration, 6) +
  model_year + factor(origin)
```

Q1 · Compute and report the in-sample fitting error (MSEin) of all the models described above. It will be easier to first write a function called mse_in(...) that returns the fitting error of a single estimated model; you can apply that function to each model (feel free to ask us for help!). We will discuss these results later.

```
# Fit all 7 required models
lm_full <- lm(cars_full, data = cars)
lm_reduced <- lm(cars_reduced, data = cars)
lm_poly2_full <- lm(cars_full_poly2, data = cars)
lm_poly2_reduced <- lm(cars_reduced_poly2, data = cars)
lm_poly6_reduced <- lm(cars_reduced_poly6, data = cars)

rt_full <- rpart(cars_full, data = cars)
rt_reduced <- rpart(cars_reduced, data = cars)

# Function to calculate in-sample MSE
mse_in <- function(model, dataset = cars) {
  predicted <- predict(model, newdata = dataset)
  actual <- dataset$mpg
  mean((actual - predicted)^2)
}

# Store models in a named list
models <- list(
  lm_full = lm_full,
  lm_reduced = lm_reduced,
  lm_poly2_full = lm_poly2_full,
  lm_poly2_reduced = lm_poly2_reduced,
  lm_poly6_reduced = lm_poly6_reduced,
  rt_full = rt_full,
  rt_reduced = rt_reduced
)

# Calculate MSEin for all models
mse_in_results <- data.frame(
  model = names(models),
  MSEin = sapply(models, mse_in)
)
```

```
mse_in_results
```

```
#>                              model     MSEin
#> lm_full                    lm_full 10.682122
#> lm_reduced              lm_reduced 10.971643
#> lm_poly2_full       lm_poly2_full  7.919030
#> lm_poly2_reduced lm_poly2_reduced  8.364546
#> lm_poly6_reduced lm_poly6_reduced  8.254377
#> rt_full                    rt_full  9.155146
#> rt_reduced              rt_reduced  9.501344
```

The model with the lowest MSEin has the best in-sample fit.

MSEin measures how well the model fits the same data it was trained on. A lower MSEin means the fitted values are closer to the actual mpg values. However, this does not guarantee the model predicts new data better, because a very complex model may overfit the training data.

Q2 · Let's try some simple evaluation of prediction error. Let's work with the lm_reduced model and test its predictive performance with split-sample testing:

(a) Split the data into 70:30 for training:test (did you remember to shuffle the data earlier?)

```
# Split data into 70% training and 30% testing
train_size <- floor(0.70 * nrow(cars))

train_data <- cars[1:train_size, ]
test_data <- cars[(train_size + 1):nrow(cars), ]
```

(b) Retrain the lm_reduced model on just the training dataset (call the new model: trained_model); Show the coefficients of the trained model.

```
# Train the reduced linear model only on the training data
trained_model <- lm(cars_reduced, data = train_data)

# Show trained model coefficients
coef(trained_model)
```

```
#>     (Intercept)          weight    acceleration      model_year factor(origin)2
#>    -25.18094941     -0.00551164      0.06855571      0.82885622      3.22722498
#> factor(origin)3
#>      2.03952491
```

(c) Use the trained_model model to predict the mpg of the test dataset. What is the in-sample mean-square fitting error (MSEin) of the trained model? What is the out-of-sample mean-square prediction error (MSEout) of the test dataset?

```
# Predict mpg for the test dataset
test_pred <- predict(trained_model, newdata = test_data)

# In-sample fitting error on training data
MSEin_train <- mean((train_data$mpg - predict(trained_model, newdata = train_data))^2)

# Out-of-sample prediction error on test data
MSEout_test <- mean((test_data$mpg - test_pred)^2)

MSEin_train
```

```
#> [1] 11.70576
```

```
MSEout_test
```

```
#> [1] 10.2117
```

The MSEin is the training error (11.70576), while the MSEout is the test prediction error (10.2117).

MSEin tells us how well the model fits the training data. MSEout tells us how well the model predicts unseen test data. Usually, MSEout is more important for prediction because it checks whether the model generalizes beyond the data it was trained on.

(d) Show a data frame of the test set's actual mpg values, the predicted mpg values, and the difference of the two (out = predictive error); Just show us the first several rows of this dataframe.

```
# Show actual, predicted, and prediction error
prediction_table <- data.frame(
  actual_mpg = test_data$mpg,
  predicted_mpg = test_pred,
  epsilon_out = test_data$mpg - test_pred
)

head(prediction_table)
```

```
#>     actual_mpg predicted_mpg epsilon_out
#> 89        14.0      14.04156 -0.04156174
#> 215       13.0      17.51041 -4.51041042
#> 288       16.5      19.40509 -2.90508936
#> 179       23.0      25.07802 -2.07801834
#> 398       31.0      29.12358  1.87642059
#> 315       26.4      26.55000 -0.14999847
```

Q3 · Let's use k-fold cross validation (k-fold CV) to see how all these models perform predictively!

(a) Write a function that performs k-fold cross-validation (see class notes and ask us online for hints!). Name your function k_fold_mse(model, dataset, k=10, ...) – it should return the MSEout of the operation. Your function must accept a model, dataset and number of folds (k) but can also have whatever other parameters you wish.

```
library(rpart)

# Put all trained models from Q1 into a list
models <- list(
  lm_full = lm_full,
  lm_reduced = lm_reduced,
  lm_poly2_full = lm_poly2_full,
  lm_poly2_reduced = lm_poly2_reduced,
  lm_poly6_reduced = lm_poly6_reduced,
  rt_full = rt_full,
  rt_reduced = rt_reduced
)

# Function for in-sample fitting error
mse_in <- function(model, dataset) {
  pred <- predict(model, newdata = dataset)
  mean((dataset$mpg - pred)^2)
}

# Function for k-fold cross-validation MSEout
k_fold_mse <- function(model, dataset, k = 10) {

  # Get the model formula
  model_formula <- formula(model)

  # Create k fold labels
  folds <- cut(1:nrow(dataset), breaks = k, labels = FALSE)

  # Store prediction errors from every fold
  pred_errors <- c()

  for (i in 1:k) {

    # Use fold i as test data
    test_indices <- which(folds == i)
    test_set <- dataset[test_indices, ]
    train_set <- dataset[-test_indices, ]
```

```
    # Refit the same type of model
    if (inherits(model, "lm")) {
      trained_model <- lm(model_formula, data = train_set)
    } else if (inherits(model, "rpart")) {
      trained_model <- rpart(model_formula, data = train_set)
    }

    # Predict test set mpg
    predictions <- predict(trained_model, newdata = test_set)

    # Actual - predicted
    errors <- test_set$mpg - predictions

    # Save errors
    pred_errors <- c(pred_errors, errors)
  }

  # Return MSEout
  mean(pred_errors^2)
}
```

The 10-fold CV MSEout reports the average prediction error across all folds for each model.

Each model is repeatedly trained on 9 folds and tested on the remaining fold. After all 10 folds are tested, the prediction errors are combined into one average MSEout. Lower MSEout means better predictive performance.

(i) Use your k_fold_mse function to find and report the 10-fold CV MSEout for all models.

```
mse_10fold <- data.frame(
  model = names(models),
  MSEout_10fold = sapply(models, k_fold_mse, dataset = cars, k = 10)
)

mse_10fold
```

```
#>                              model MSEout_10fold
#> lm_full                    lm_full     11.262460
#> lm_reduced              lm_reduced     11.415855
#> lm_poly2_full        lm_poly2_full      8.599373
#> lm_poly2_reduced  lm_poly2_reduced      8.818607
#> lm_poly6_reduced  lm_poly6_reduced      9.267369
#> rt_full                    rt_full     13.342221
#> rt_reduced              rt_reduced     13.476272
```

(ii) For all the models, which is bigger — the fit error (MSEin) or the prediction error (MSEout)? (optional: why do you think that is?)

```
mse_fit <- data.frame(
  model = names(models),
  MSEin = sapply(models, mse_in, dataset = cars)
)

compare_10fold <- merge(mse_fit, mse_10fold, by = "model")

compare_10fold$larger_error <- ifelse(
  compare_10fold$MSEout_10fold > compare_10fold$MSEin,
  "MSEout is bigger",
  "MSEin is bigger"
)

compare_10fold
```

```
#>              model     MSEin MSEout_10fold     larger_error
#> 1          lm_full 10.682122     11.262460 MSEout is bigger
#> 2    lm_poly2_full  7.919030      8.599373 MSEout is bigger
#> 3 lm_poly2_reduced  8.364546      8.818607 MSEout is bigger
#> 4 lm_poly6_reduced  8.254377      9.267369 MSEout is bigger
#> 5       lm_reduced 10.971643     11.415855 MSEout is bigger
#> 6          rt_full  9.155146     13.342221 MSEout is bigger
#> 7       rt_reduced  9.501344     13.476272 MSEout is bigger
```

For all models here, the prediction error MSEout is larger than the fit error MSEin.

This happens because MSEin is calculated using data the model has already seen, while MSEout is calculated using held-out data. A model usually fits training data better than unseen data. If the gap is very large, it may suggest overfitting.

(iii) Does the 10-fold MSEout of a model remain stable (same value) if you re-estimate it over and over again, or does it vary? (show results of a few repetitions for any model and decide!)

```
repeat_10fold <- replicate(
  5,
  k_fold_mse(lm_reduced, cars, k = 10)
)

repeat_10fold
```

```
#> [1] 11.41586 11.41586 11.41586 11.41586 11.41586
```

In my current function, the 10-fold MSEout stays the same because the fold assignment is fixed. However, if the folds were randomly reassigned each repetition, the 10-fold MSEout would vary slightly.

If the function creates the same folds every time, then the same rows are used for training and testing, so the result is stable. If the rows are randomly split into folds each repetition, the training and test sets change slightly, so the MSEout can also change slightly.

(b) Make sure your k_fold_mse() function can accept as many folds as there are rows (i.e., k=392).

(i) How many rows are in the training dataset and test dataset of each iteration of k-fold CV when k=392?

```
n <- nrow(cars)

training_rows <- n - 1
test_rows <- 1

training_rows
```

```
#> [1] 391
```

```
test_rows
```

```
#> [1] 1
```

When k = 392, each iteration uses 391 rows for training and 1 row for testing.

This is called leave-one-out cross-validation. Since there are 392 rows, the model leaves out one row as the test set and trains on all remaining rows. This repeats until every row has been tested once.

(ii) Report the k-fold CV MSEout for all models using k=392.

```
mse_392fold <- data.frame(
  model = names(models),
  MSEout_392fold = sapply(models, k_fold_mse, dataset = cars, k = nrow(cars))
)

mse_392fold
```

```
#>                            model MSEout_392fold
#> lm_full                  lm_full      11.293439
#> lm_reduced            lm_reduced      11.380040
#> lm_poly2_full       lm_poly2_full       8.610385
#> lm_poly2_reduced lm_poly2_reduced       8.787013
#> lm_poly6_reduced lm_poly6_reduced       9.177932
#> rt_full                  rt_full      12.769791
```

```
#> rt_reduced             rt_reduced      13.145150
```

The k = 392 CV MSEout reports the leave-one-out prediction error for all models.

This is a stricter version of cross-validation because every individual row gets tested separately. The final MSEout is the average squared prediction error across all 392 left-out predictions.

(iii) When k=392, does the MSEout of a model remain stable (same value) if you re-estimate it over and over again, or does it vary? (show a few repetitions for any model and decide!)

```
repeat_392fold <- replicate(
  5,
  k_fold_mse(lm_reduced, cars, k = nrow(cars))
)

repeat_392fold
```

```
#> [1] 11.38004 11.38004 11.38004 11.38004 11.38004
```

When k = 392, the MSEout should remain stable if the dataset order and model settings stay the same.

With leave-one-out CV, each row is used as the test set exactly once. Because the test folds are fixed as single rows, repeating the process should give the same result unless the model itself contains randomness.

(iv) Looking at the fit error (MSEin) and prediction error (MSEout; k=392) of the full models versus their reduced counterparts (with the same training technique), does multicollinearity present in the full models seem to hurt their fit error and/or prediction error? (optional: if not, then when/why are analysts so scared of multicollinearity?)

```
final_compare <- Reduce(
  function(x, y) merge(x, y, by = "model"),
  list(mse_fit, mse_10fold, mse_392fold)
)

final_compare[
  final_compare$model %in% c(
    "lm_full", "lm_reduced",
    "lm_poly2_full", "lm_poly2_reduced",
    "rt_full", "rt_reduced"
  ),
]
```

```
#>              model     MSEin MSEout_10fold MSEout_392fold
#> 1          lm_full 10.682122     11.262460      11.293439
#> 2    lm_poly2_full  7.919030      8.599373       8.610385
```

```
#> 3 lm_poly2_reduced  8.364546      8.818607       8.787013
#> 5       lm_reduced 10.971643     11.415855      11.380040
#> 6          rt_full  9.155146     13.342221      12.769791
#> 7       rt_reduced  9.501344     13.476272      13.145150
```

The full models have slightly lower MSEin and MSEout than their reduced counterparts, so multicollinearity does not seem to hurt fit error or prediction error in this dataset.

Multicollinearity mainly makes regression coefficients unstable and harder to interpret. It may change coefficient sizes, standard errors, and p-values. However, prediction can still be good if the correlated variables contain similar useful information. So analysts worry about multicollinearity more when they care about interpretation, not only prediction.

(v) Look at the fit error and prediction error (k=392) of the reduced quadratic versus 6th order polynomial regressions — did adding more higher-order terms hurt the fit and/or predictions? (optional: What does this imply? Does adding complex terms improve fit or prediction?)

```
final_compare[
  final_compare$model %in% c("lm_poly2_reduced", "lm_poly6_reduced"),
]
```

```
#>              model    MSEin MSEout_10fold MSEout_392fold
#> 3 lm_poly2_reduced 8.364546      8.818607       8.787013
#> 4 lm_poly6_reduced 8.254377      9.267369       9.177932
```

If the 6th-order polynomial lowers MSEin but does not lower MSEout, then adding more complex terms improved fit but not prediction.

Higher-order polynomial terms make the model more flexible, so it can fit the training data more closely. But if the prediction error does not improve, the extra complexity may be overfitting noise instead of learning a useful pattern. This means complex models do not always predict better.